СЕТЕВОЙ УРОВЕНЬ
Часть III

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

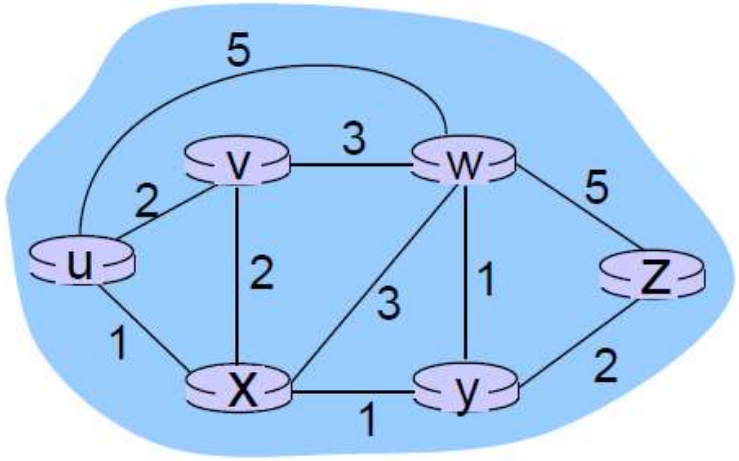
Алгоритмы маршрутизации

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Взаимодействие между маршрутизацией и перенаправлением

Абстракция графа



граф: G = (N,E)

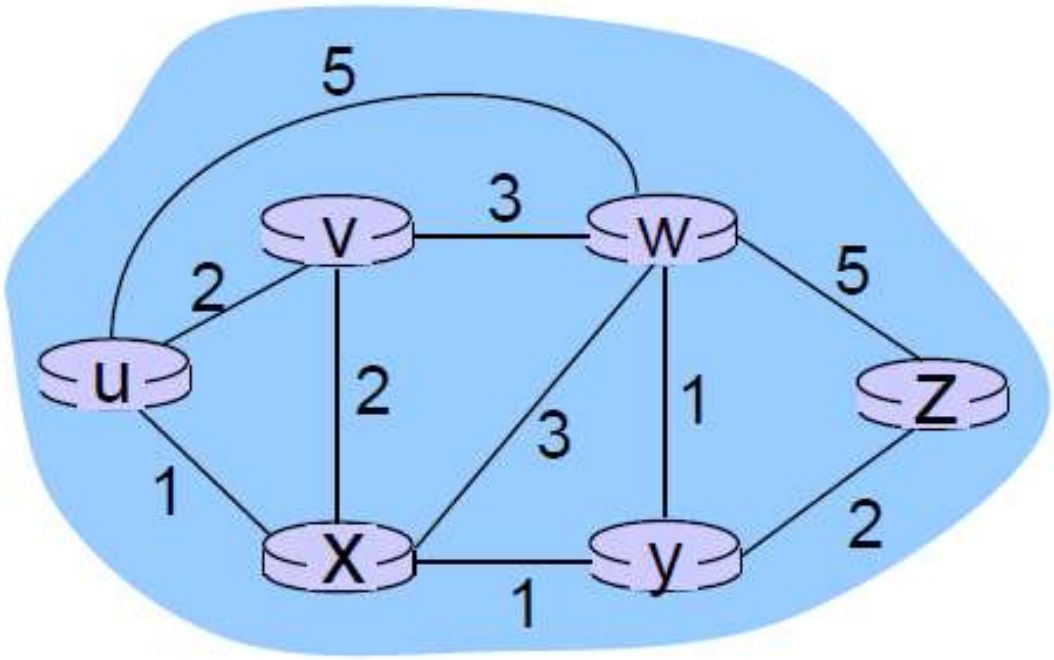
N = множество маршрутизаторов = { u, v, w, x, y, z }

E = множество каналов ={ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) }

Вес ребра отражает физическую длину соответствующего канала в сети

Примечание: абстракция графа удобна и в других сетевых контекстах, например, P2P, где ***N*** – множество партнеров, а ***E*** – множество TCP-соединений

Абстракция графа: стоимости



c(x,x') = стоимость канала *(x,x')*

Например, *c(w,z) = 5*

Стоимость всегда может быть равна либо быть обратно пропорциональна
ширине полосы, либо обратно пропорциональна перегрузке

Стоимость пути *(x1, x2, x3,..., xp) = c(x1,x2) + c(x2,x3) + … + c(xp-1,xp)*

Классификация алгоритмов маршрутизации

Вопрос: **глобальная** или **децентрализованная** информация?

Глобальная:

- Все маршрутизаторы обладают подробной информацией о топологии, стоимости каналов
- Алгоритмы, «учитывающие состояние канала»

Децентрализованные:

- Маршрутизатору известны физически подключенные к нему соседние устройства, стоимости каналов до них
- Итеративный вычислительный процесс, обмен информацией с соседними устройствами
- «дистанционно-векторные» алгоритмы

Вопрос: статические или динамические?

Статические:

- Маршруты мало меняются со временем

Динамические:

- Маршруты меняются быстрее
  - периодические обновления в ответ на изменение стоимостей каналов

Современные алгоритмы маршрутизации, действующие в Интернете (в частности, RIP, OSPF и BGP), **нечувствительны к нагрузкам**. Это означает, что стоимость канала не находится в прямой зависимости от того, насколько он загружен

Алгоритм маршрутизации с учетом состояния каналов

Link-state algorithm, LS

Алгоритм Дейкстры

- Топология сети, стоимости каналов известны всем узлам
  - Реализуется при помощи «широковещательного сообщения о состоянии канала» всем другим узлам в сети
  - У всех узлов одна и та же информация
- Вычисляет пути с наименьшей стоимостью от одного узла («источник») до всех остальных
  - дает таблицу перенаправления для данного узла
- Итеративный: после **k** итераций узнает путь наименьшей стоимости к k узлам назначения

Нотация:

- c(x,y): стоимость канала от узла **x** к **y**;
- =∞, если узлы не являются непосредственно смежными
- D(v): текущее значение стоимости пути от источника к узлу назначения **v**
- p(v): предшествующий узел на пути от источника к **v**
- N': множество узлов, пути наименьшей стоимости до которых точно известны

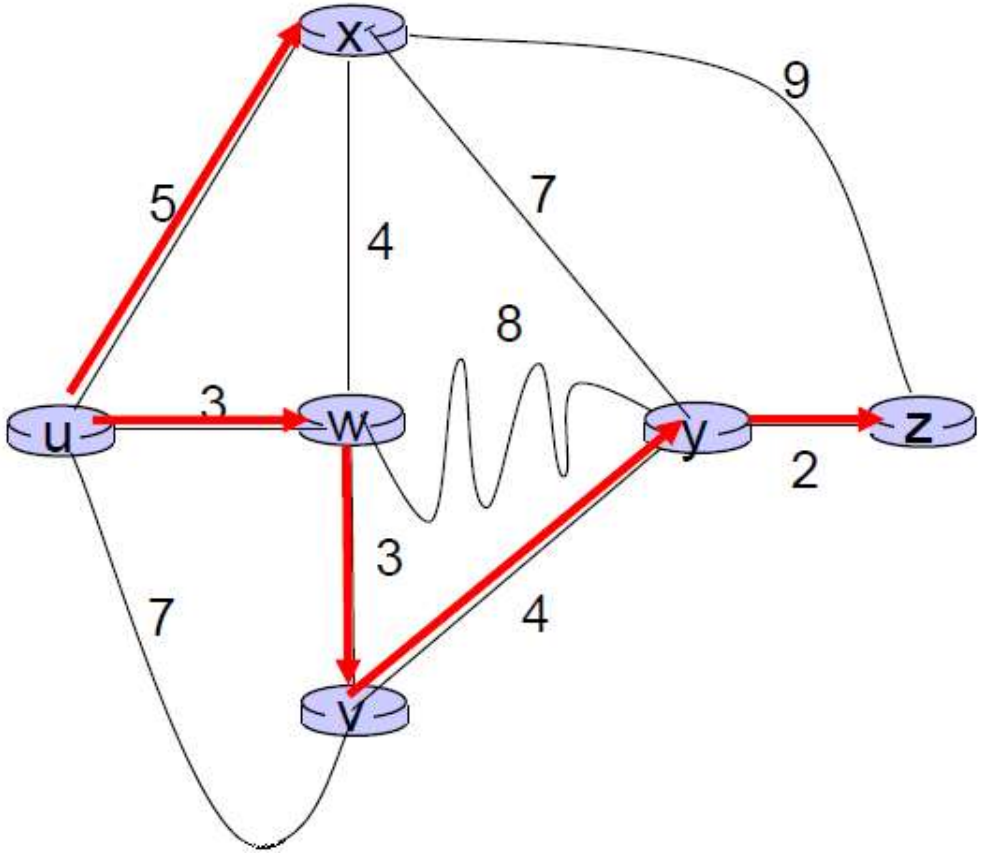
Алгоритм Дейкстры

```
Инициализация:
  N' = {u}
  для всех узлов v
    if v смежный с u
        then D(v) = c(u,v)
    else D(v) = ∞

Цикл
  Найти такое w, не принадлежащее к N‘, чтобы D(w) было минимальным
  прибавить w к N'
  обновить D(v) для всех v смежных с w и не относящихся к N' :
      D(v) = min( D(v), D(w) + c(w,v) )
  /* новая стоимость до v – это либо предыдущая стоимость до v или известная
  стоимость кратчайшего пути до w плюс стоимость от w до v */
пока не будут перебраны все узлы в N'
```

Алгоритм Дейкстры: пример

Шаг	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

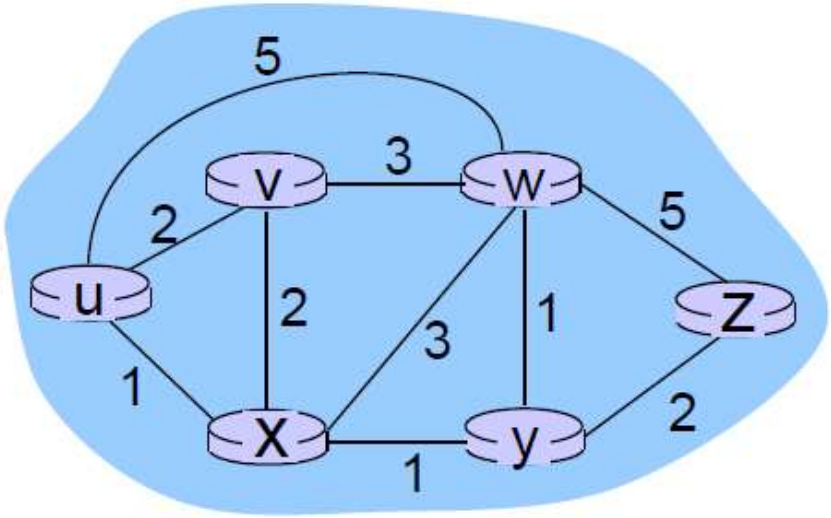


Примечание:

- Составить дерево кратчайших путей, отследив предшествующие узлы

Алгоритм Дейкстры: пример

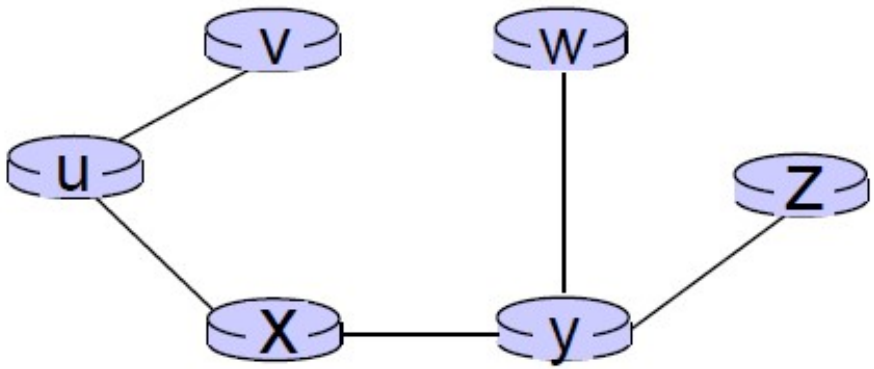
Шаг	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



Факультет математики и компьютерных наук

Алгоритм Дейкстры: пример

- Результирующее дерево кратчайших путей от **u**:



- Результирующая таблица перенаправления на узле **u**:

узел назначения	канал
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

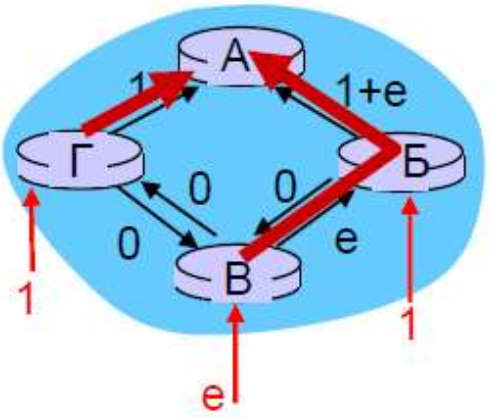
Алгоритм Дейкстры: обсуждение

Сложность алгоритма: *n* узлов

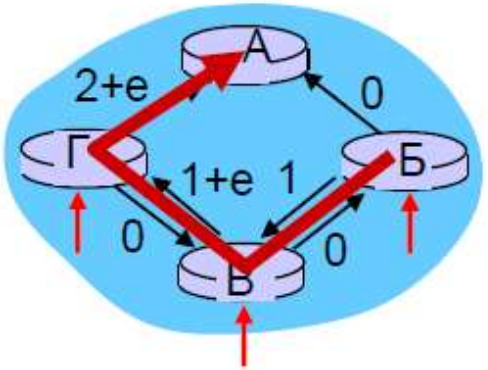
- Каждая итерация: необходимо проверить все узлы *w*, не относящиеся к N
- $n(n+1)/2$ сравнений: $O(n$^2)
- Возможны более эффективные реализации: $O(n log n)$

Возможные проблемы:

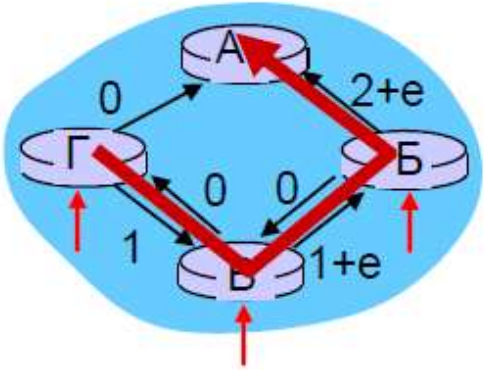
- например, стоимость поддержки канала равна общему объему переносимого по нему трафика:



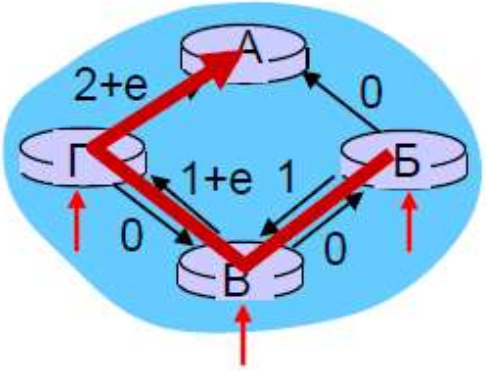
изначально



Имея эти значения, найти новый маршрут…. получаем новые стоимости



Имея эти значения, найти новый маршрут…. получаем новые стоимости



Имея эти значения, найти новый маршрут…. получаем новые стоимости

Дистанционно-векторный алгоритм

Алгоритм является:

- Итерационным. Процесс продолжается до тех пор, пока узлы-соседи полностью не обменяются той или иной информацией
- Асинхронным
- Распределенным

Уравнение Беллмана-Форда (динамическое программирование)

пусть

$d_x(y)$ = стоимость кратчайшего пути от x до y

тогда

$$d_x(y) = \min_v \{c(x,v) + dv(y)\}$$

Стоимость пути от смежного v до конечного узла y

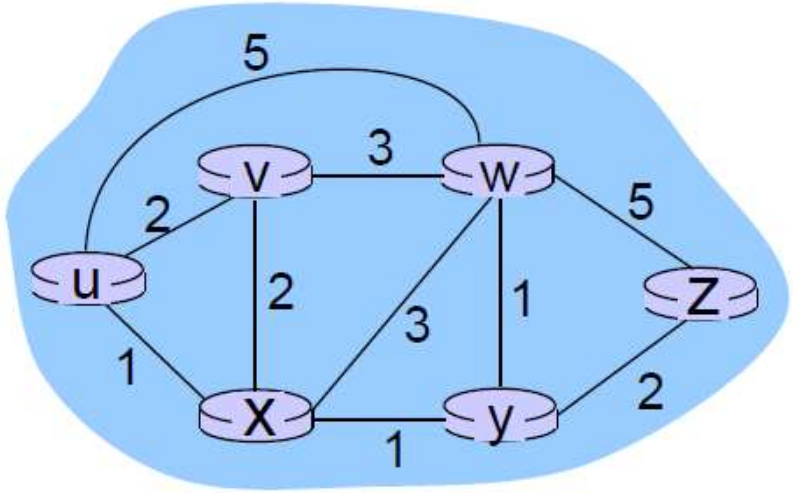
Стоимость пути до смежного узла v

min значение, взятое для всех узлов v смежных с x

Пример уравнения Беллмана-Форда

- Очевидно, dv(z) = 5, dx(z) = 3, dw(z) = 3
- Согласно уравнению Б-Ф:

$$
\begin{aligned}
d_u(z) = \min\{ & c(u,v) + dv(z), \\
& c(u,x) + dx(z), \\
& c(u,w) + d_w(z) \} \\
= \min\{ & 2 + 5, \\
& 1 + 3, \\
& 5 + 3\} = 4
\end{aligned}
$$



Узел, достигающий минимального значения, находящийся на расстоянии одного перехода по кратчайшему пути, находится в таблице перенаправления

Дистанционно-векторный алгоритм

- $D_x(y)$= оценка наименьшей стоимости от **x** до **y**
  - **x** поддерживает дистанционно-векторную информацию

$$\boldsymbol{D}_x = [D_x(y): y \in N]$$

- Узел **x**:
  - Знает стоимость пути до каждого соседа **v**: $c(x,v)$
  - Поддерживает информацию о дистанционных векторах своих соседних узлов. Для каждого соседа **v** узел **x** поддерживает

$$\boldsymbol{D}_v = [D_v(y): y \in N]$$

Дистанционно-векторный алгоритм

Основная идея:

- Время от времени каждый узел посылает смежным узлам свою оценку дистанционного вектора
- Когда **x** получает новую оценку DV от соседнего узла, он обновляет собственный DV при помощи уравнения Б-Ф:

$$D_x(y) \leftarrow min_v\{c(x,v) + D_v(y)\} \text{ для каждого узла } y \in N$$

- В менее распространенных естественных условиях оценки $D_x(y)$ сходятся к фактическому минимальному значению $d_x(y)$

Дистанционно-векторный алгоритм

Итеративный, асинхронный: каждая локальная итерация обусловлена:

- Локальным изменением стоимости канала
- Сообщение об обновлении DV, полученное от соседнего узла

Распределенный:

- Каждый узел уведомляет соседние, лишь если у него обновляется DV
- Затем его соседи при необходимости уведомляют свои соседние узлы

Каждый узел:

ожидает (изменение локальной стоимости канала или сообщения от соседа)

↓

пересчитывает оценки

↓

если DV до любого узла назначения изменился, *уведомляет* соседей

$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

Таблица узла x

от \ стоим. до	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

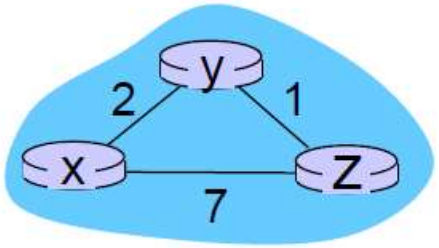
Таблица узла y

от \ стоим. до	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

Таблица узла z

от \ стоим. до	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

from \ стоим. до	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0



время

$D_x(y) = \min\{c(x,y) + D_y(y),\ c(x,z) + D_z(y)\}$
$= \min\{2+0\ ,\ 7+1\} = 2$

$D_x(z) = \min\{c(x,y) + D_y(z),\ c(x,z) + D_z(z)\}$
$= \min\{2+1\ ,\ 7+0\} = 3$

Таблица узла x

от \ стоим. до	x	y	z
x	0	2	7
y	∞	∞	∞
z	∞	∞	∞

от \ стоим. до	x	y	z
x	0	2	3
y	2	0	1
z	7	1	0

от \ стоим. до	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

Таблица узла y

от \ стоим. до	x	y	z
x	∞	∞	∞
y	2	0	1
z	∞	∞	∞

от \ стоим. до	x	y	z
x	0	2	7
y	2	0	1
z	7	1	0

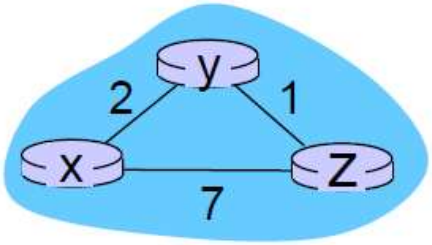
от \ стоим. до	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

Таблица узла z

от \ стоим. до	x	y	z
x	∞	∞	∞
y	∞	∞	∞
z	7	1	0

от \ стоим. до	x	y	z
x	0	2	7
y	2	0	1
z	3	1	0

от \ стоим. до	x	y	z
x	0	2	3
y	2	0	1
z	3	1	0

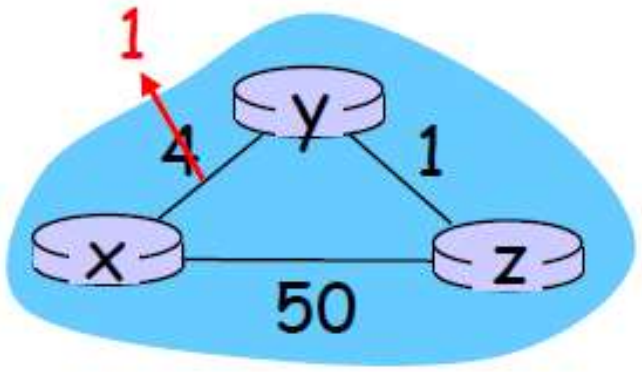


время

Дистанционный вектор: изменение стоимости каналов

Изменение стоимости каналов:

- Узел обнаруживает локальное изменение стоимости канала
- Обновляет информацию о маршрутизации, пересчитывает дистанционный вектор. Если DV изменяется, уведомляет об этом соседей

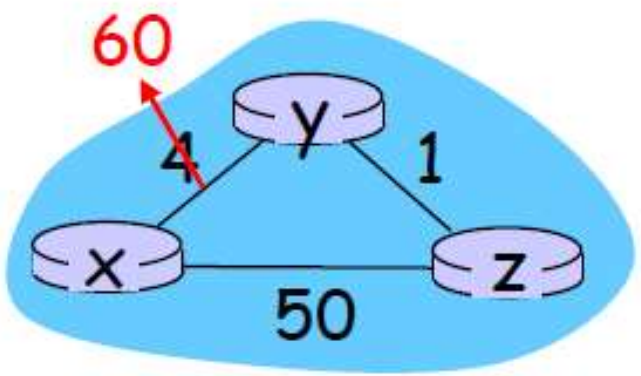


- t0 : **y** обнаруживает изменение стоимости канала, обновляет свой DV, информирует об этом соседей
- t1 : **z** получает обновление от **y**, обновляет свою таблицу, вычисляет новый кратчайший путь до **x**, рассылает соседям свой DV
- t2 : **y** получает обновление от **z**, обновляет свою таблицу расстояний. Наименьшие стоимости для **y** не меняются, поэтому **y** не посылает сообщение к **z**

Дистанционный вектор: изменение стоимости канала

Изменение стоимости канала:

- Узел обнаруживает изменение локальной стоимости канала
- Плохие новости идут медленно – проблема «счета до бесконечности»
- 44 итерации проходит до того, как алгоритм стабилизируется



Отравление обратного пути:

- Если **Z** направляет трафик через **Y**, чтобы попасть в **X**:
  - **Z** сообщает **Y**, что расстояние от него (Z) до **X** является бесконечным (поэтому **Y** не будет направлять трафик в **X** через **Z**)
- Можно ли таким образом полностью решить проблему счета до бесконечности?

Сравнение алгоритма с учетом состояния каналов (LS) и дистанционно-векторный алгоритм (DV)

Сложность сообщения

- LS: зависит от n узлов, E каналов, $O(nE)$ отправленных сообщений
- DV: обмен информацией только между соседями
  - Время схождения варьируется

Скорость схождения

- LS: алгоритм $O(n$^$\mathbf{2})$ требует $O(nE)$ сообщений
  - Может иметь колебания
- DV: время схождения отличается
  - Могут возникать маршрутные петли
  - Проблема счета до бесконечности

Отказоустойчивость: что происходит при сбоях маршрутизатора?

LS:

- Узел может объявить неверную стоимость канала
- Каждый узел вычисляет только свою собственную таблицу

DV:

- Узел DV может объявить неверную стоимость пути
- Таблица каждого узла используется другими узлами
  - Ошибка распространяется по сети

Иерархическая маршрутизация

До сих пор мы изучали маршрутизацию в идеализированном виде

- Все маршрутизаторы идентичны
- Сеть «плоская»

НО на практике все иначе

Масштаб: около 600 миллионов хостов назначения:

- Невозможно сохранить все места назначения в таблицах перенаправления!
- Информация, которой обмениваются таблицы перенаправления, забивает каналы!

Административная автономия

- Интернет = сеть сетей
- Каждому сетевому администратору может потребоваться управлять маршрутизацией во вверенной ему сети

Иерархическая маршрутизация

- Объединение маршрутизаторов в области - автономные системы (АС)
- Маршрутизаторы в одной АС используют один и тот же протокол маршрутизации
  - Протокол «внутренней» маршрутизации
  - Маршрутизаторы в разных АС могут использовать разные протоколы внутренней маршрутизации

Шлюзовой (граничный) маршрутизатор:

- Стоит на «границе» своей АС
- Связан каналом с маршрутизатором из другой АС

Взаимосвязанные АС

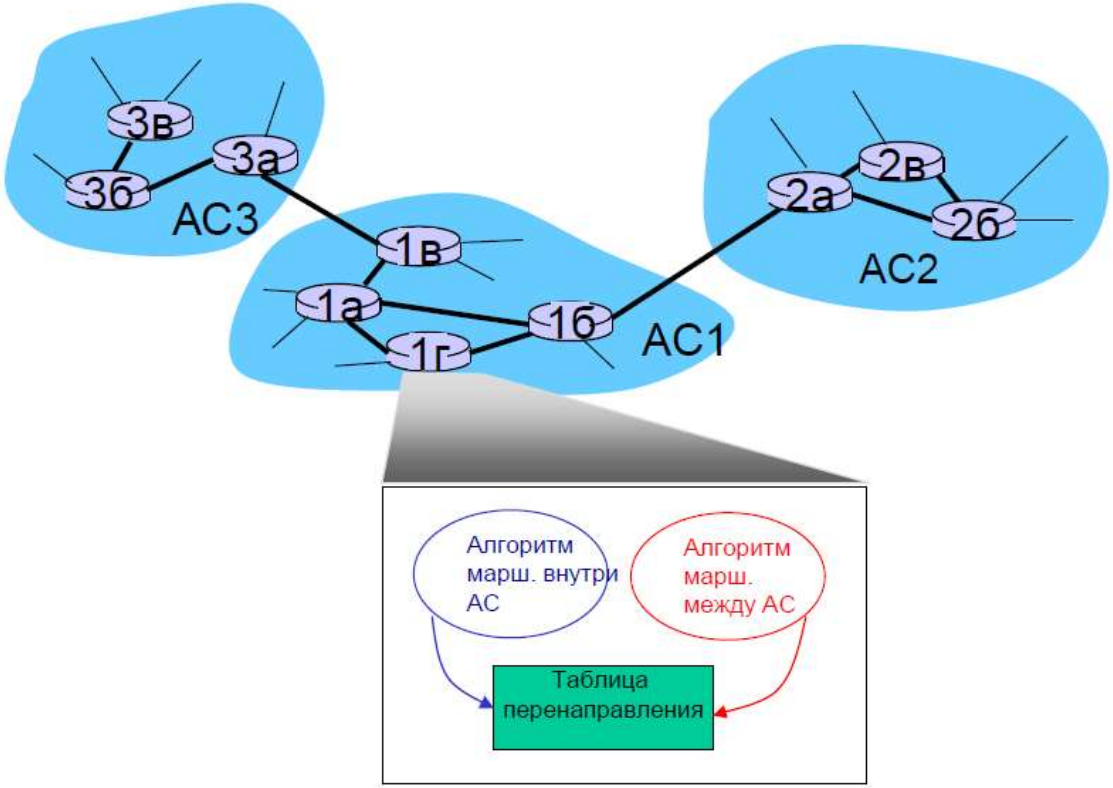


Таблица перенаправления конфигурируется с учетом алгоритмов маршрутизации, действующих как внутри АС, так и между ними

- Внутренний алгоритм задает записи для мест назначения внутри системы
- Внутренний и внешний алгоритмы задают записи для внешних мест назначения

Задачи, решаемые между АС

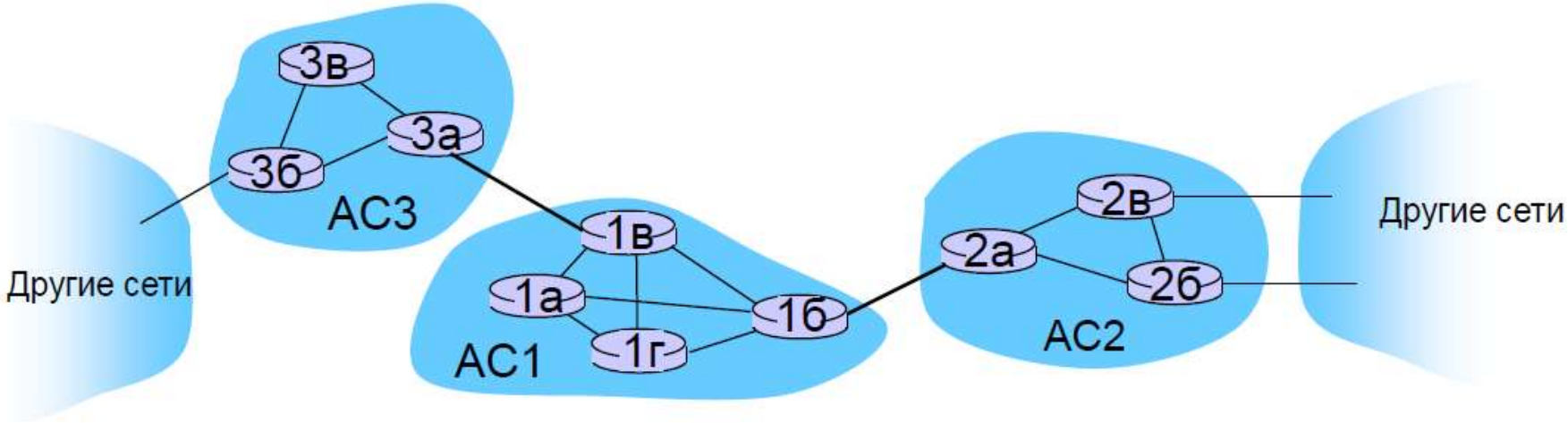
Предположим, маршрутизатор в **АС1** получает дейтаграмму, идущую за пределы **АС1**:

- Маршрутизатор должен передать этот пакет на шлюзовой маршрутизатор, но на который?

АС1 должна:

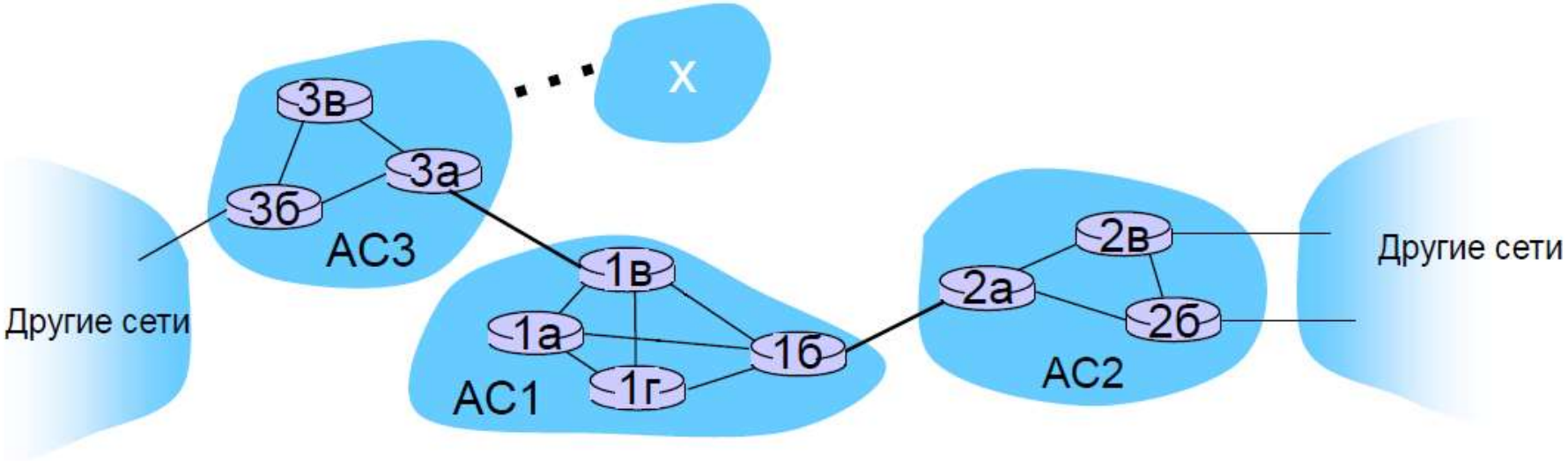
1. Узнать, какие места назначения достижимы через **АС2**, какие – через **АС3**
2. Распространить эту информацию между всеми маршрутизаторами в **АС1**

Задача внешней маршрутизации!



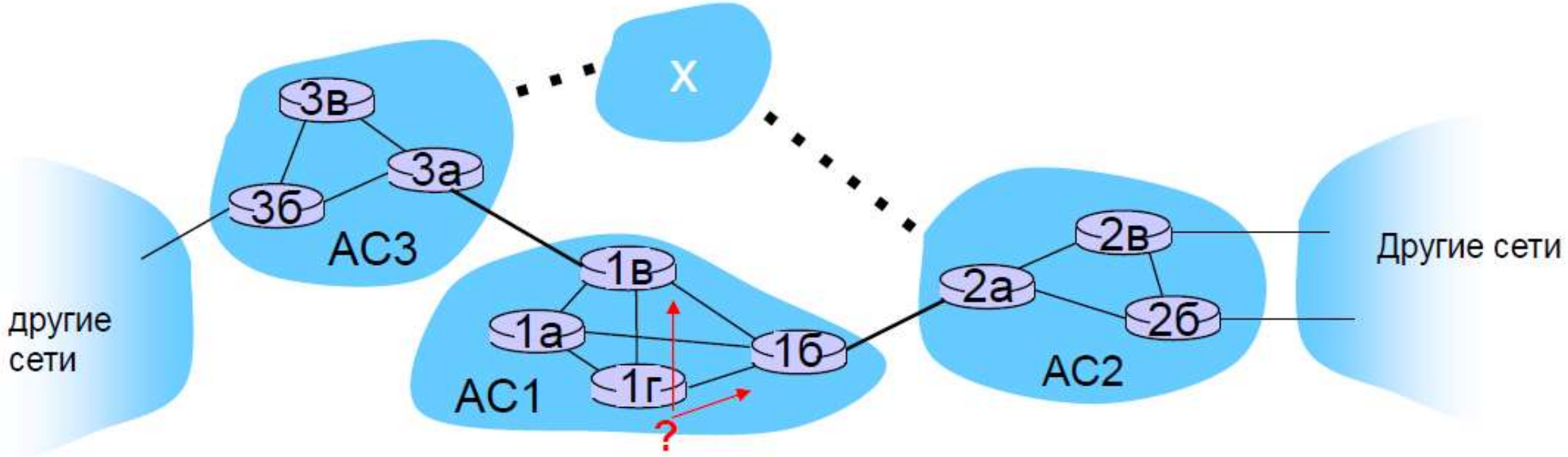
Пример: настройка таблицы перенаправления в маршрутизаторе **1г**

- Допустим, система *АС1* узнает (по внешнему протоколу), что подсеть **x** достижима через *АС3* (шлюз **1в**), но не через *АС2*
  - Внешний протокол распространяет эту информацию о достижимости между всеми внутренними маршрутизаторами в данной АС
  - По алгоритму внутренней маршрутизации маршрутизатор **1г** определяет, что его интерфейс **I** находится на кратчайшем пути к **1в**
  - Заносит в таблицу перенаправления запись **(x, I)**



Пример: выбор одной АС из нескольких

- Далее предположим, что *АС1* узнает из внешнего протокола, что подсеть **x** достижима как из *АС3*, так и из *АС2*
- Чтобы сконфигурировать таблицу перенаправления, маршрутизатор **1г** должен определить, через какой шлюз пересылать пакеты к месту назначения **x**
  - Это также задача протокола внешней маршрутизации
  - Работает принцип «горячей картофелины» (hot-potato routing)



Пример: выбор одной АС из нескольких

Маршрутизация по **принципу горячей картофелины**: послать пакет к ближайшему из двух маршрутизаторов

Узнает из внешнего протокола, что подсеть *x* достижима через несколько шлюзов

→

Использует информацию о маршрутизации из внутреннего протокола для определения кратчайших путей к каждому из шлюзов

→

Маршрутизация по принципу горячей картофелины: выбираем тот шлюз, кратчайший путь до которого самый дешевый

→

Из таблицы перенаправления узнаем интерфейс *I*, ведущий к шлюзу кратчайший путь до которого самый дешевый Записываем *(x,I)* в таблицу перенаправления

Маршрутизация в сети Интернет

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

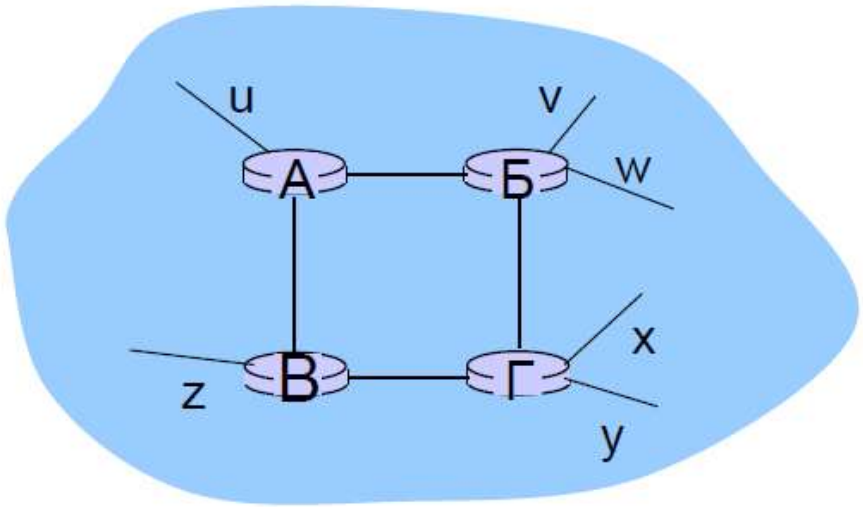
Маршрутизация внутри АС

- Также известен термин протоколы внутреннего шлюза (Interior Gateway Protocol, IGP)

- Наиболее распространенные протоколы маршрутизации, действующие внутри АС:
  - **RIP** (Routing Information Protocol): протокол маршрутной информации
  - **OSPF** (Open Shortest Path First): протокол нахождения кратчайшего пути
  - **IGRP**: протокол маршрутизации внутреннего шлюза (запатентован Cisco)

RIP (протокол маршрутной информации)

- Routing Information Protocol, RIP
- Включен в дистрибутив BSD-UNIX в 1982 году
- Дистанционно-векторный алгоритм
  - Измерение расстояния: # переходов (max = 15 переходов), стоимость каждого канала равна 1. Это ограничивает максимальный диаметр АС
  - Обмен дистанционными векторами с соседями происходит каждые 30 с. В ответном сообщении (также называемом объявлением)
  - Каждое объявление – это список до 25 подсетей назначения (в контексте IP-адресации)



От маршрутизатора А к *подсетям* назначения:

подсеть	переходы
u	1
v	2
w	2
x	3
y	3
z	2

Протокол RIP: пример

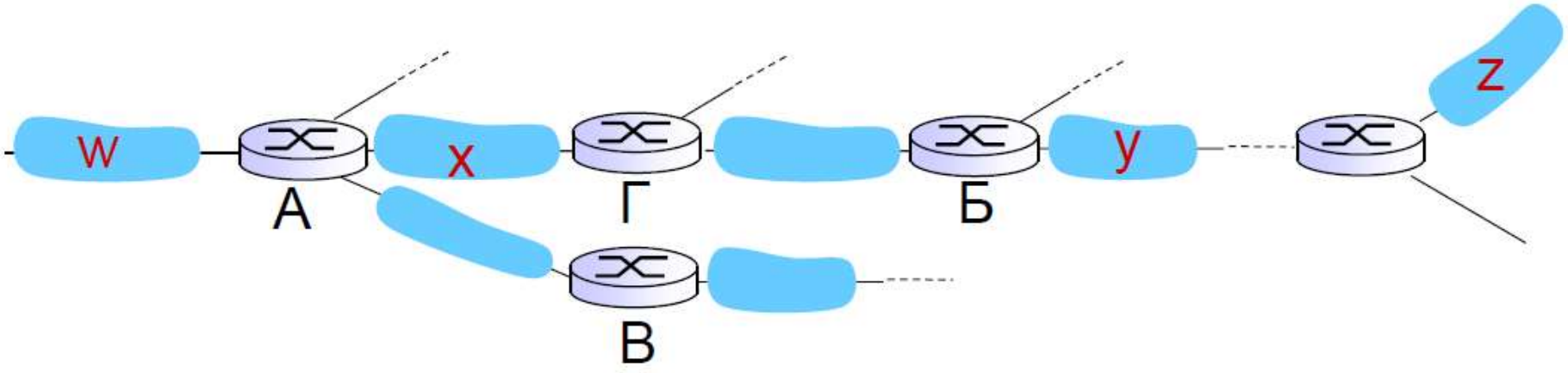


Таблица перенаправления в маршрутизаторе Г

Подсеть назначения	след. маршрутизатор	кол-во переходов
w	А	2
y	Б	2
z	Б	7
x	--	1
....	....	....

Протокол RIP: пример

Предположим, что 30 с спустя маршрутизатор **Г** получает от маршрутизатора **А** новое объявление

Сеть **z** теперь находится на расстоянии всего четырех переходов от маршрутизатора **А**

Объявление от А к Г

назн.	след.	перех.
w	-	1
x	-	1
z	B	4
....	....	....

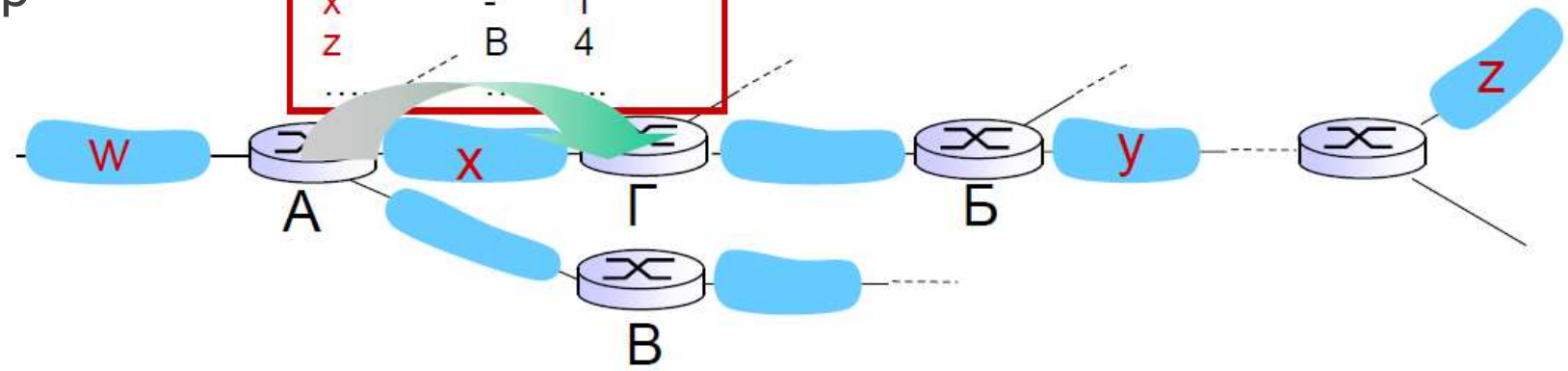


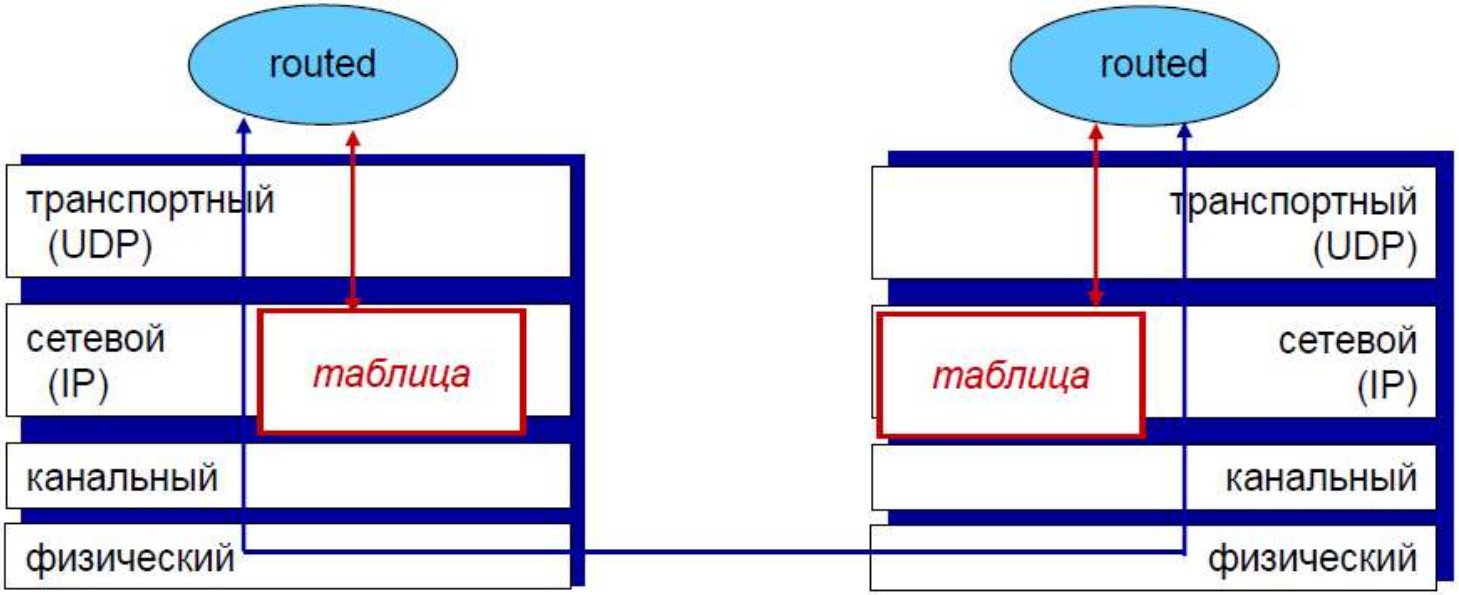
Таблица перенаправления в маршрутизаторе Г

Подсеть назначения	след. маршрутизатор	кол-во переходов
w	А	2
y	Б	2
z	~~Б~~ А	~~7~~ 5
x	--	1
....	....	....

Протокол RIP: неисправности канала, восстановление

Если за 180 с не получено ни одного объявления --> смежный узел/канал считаются отказавшими

- Маршруты через этот узел упраздняются
- Соседям посылаются новые объявления
- Информация об отказе канала распространяется по всей сети
- Отравление обратного пути используется для предотвращения пинг-понгового зацикливания (бесконечное расстояние = 16 переходов)

Обработка RIP-таблицы

- Таблицы маршрутизации RIP управляются процессом (демоном) прикладного уровня, называемым *routed*
- Объявления пересылаются в UDP-пакетах, периодически повторяются
- Протокол RIP представляет собой **протокол прикладного уровня**, работающий поверх протокола UDP